



Meridian

— Prepared by Chris

Chris

ra3964@nyu.edu

01 — Project Overview

Meridian is a Solana-based token authorization and x402 payment-settlement system that associates SPL-compatible token vaults with Ethereum addresses and enables transfers through secp256k1-signed spending permits. It supports deposits, delegated transfers within authorized amount limits, replay protection through single-use nonces, and nonce cancellation. Its x402 settlement layer allows facilitators to execute exact, recipient-bound payments from funded vaults once a payer-specified validity time has been reached.

02 — Scope

Name	Description
<code>permit2</code>	Solana program <code>permit2</code> under audit.
<code>x402</code>	Solana program <code>x402</code> under audit.

03 — Findings

Severity	Count
CRITICAL	1
HIGH	2
MEDIUM	0
LOW	0
INFO	0

04 — Vulnerabilities

ID	Severity	Description
SP-MER-ADV-00	CRITICAL	A length-only check accepts unrelated 32-byte messages, allowing historical signatures to authorize vault withdrawals.
SP-MER-ADV-01	HIGH	Signed permit deadlines are never validated, allowing unspent authorizations to withdraw vault tokens indefinitely.
SP-MER-ADV-02	HIGH	Fully consumed nonce bitmaps can be closed and recreated, allowing spent or invalidated permits to be replayed.

Permit Signatures Are Not Bound to Message Hashes

CRITICAL

SP-

MER-ADV-00

Description

`verify_eth_signature` is intended to prove that the vault owner signed the `data_hash` derived from the current permit. Although the native `Secp256k1Program` verifies the signature against the message embedded in its instruction, `parse_secp256k1_instruction` only compares that message's length with `data_hash`. Because `data_hash` is always 32 bytes, any precompile-verified 32-byte message is accepted without its contents being compared. Consequently, any historical 32-byte secp256k1 signature produced by the vault owner can authorize caller-supplied permit parameters, including the token, amount, nonce, deadline, spender, recipient, and witness. The account and argument checks in `permit_transfer_from_common` only establish consistency among caller-supplied values and do not cryptographically bind them to the signature. Unused nonce bits can also be consumed repeatedly across nonce words, allowing current balances and subsequent vault top-ups to be swept on-chain through `permit2` and `x402::settle`.

```
// programs/permit2/src/crypto.rs
fn parse_secp256k1_instruction(data: &[u8], self_index: u16, data_hash: &[u8; 32]) -
> Result<Secp256k10ffsets> {
    // ...
    let message = data[message_offset..message_offset + message_size].to_vec();

    require!(
        data_hash.len() == message.len(),
        Permit2Error::InvalidSignature
    );
    // ...
}
```

Remediation

Replace the length-only comparison with `message.as_slice() == data_hash` so the precompile-verified message must exactly match the permit's domain-specific hash.

Signed Permit Deadlines Are Not Enforced

HIGH

SP-MER-ADV-01

Description

The `deadline` field in `PermitTransferFromArgs` is incorporated into both signed permit hashes, indicating that authorization is intended to remain valid only through that timestamp. However, `permit_transfer_from_common` does not read `Clock::sysvar` or compare the current timestamp against `permit.deadline`, despite `Permit2Error::SignatureExpired` being defined.

Both `permit_transfer_from` and `permit_witness_transfer_from` rely on the affected common function. Although `x402::settle` validates `witness.valid_after`, this establishes only an earliest execution time and does not enforce the permit's latest execution time.

Consequently, any signed permit with an unused `nonce` remains executable indefinitely by its designated `spender` or `x402` facilitator. For example, a permit for `100_000_000` tokens with `deadline = 1_700_000_000` can still be settled at timestamp `1_700_000_001`, reducing a vault balance from `500_000_000` to `400_000_000` after authorization expiry.

```
// programs/permit2/src/instructions.rs
pub fn permit_transfer_from_common(
    ctx: Context<PermitTransfer>,
    permit: PermitTransferFromArgs,
    transfer_details: SignatureTransferDetails,
    eth_address: [u8; 20],
    data_hash: [u8; 32],
) -> Result<()> {
    require!(
        transfer_details.requested_amount <= permit.permitted_amount,
        Permit2Error::InvalidAmount
    );
    // ...
```

Remediation

Read `Clock::get()?.unix_timestamp` at the beginning of `permit_transfer_from_common`, before consuming the nonce. Reject negative timestamps and timestamps greater than `permit.deadline` with `Permit2Error::SignatureExpired`.

Permissionless Bitmap Closure Reenables Permit Replay HIGH SP-

MER-ADV-02

Description

The `NonceBitmap` PDA is the only persistent on-chain record of spent or invalidated nonces for each `(eth_address, word_pos)` pair. Once both `word_lo` and `word_hi` equal `u128::MAX`, the account may be deleted through `close_nonce_bitmap`.

The `CloseNonceBitmap` context contains no `Signer<'info>` or owner authorization and can therefore be invoked permissionlessly. Its `close = rent_payer` constraint deletes the bitmap without recording that the nonce word was finalized. Subsequent calls to `permit_transfer_from` or `invalidate_unordered_nonces` recreate the same PDA through `init_if_needed`, with both bitmap halves reset to zero; this fresh state is then accepted by `use_unordered_nonce`.

A word may become closable through ordinary permit usage or an owner-signed invalidation that fills its remaining bits. After closure, all 256 nonces are restored, so a previously spent or explicitly invalidated permit can be resubmitted to transfer vault funds again.

```
// programs/permit2/src/context.rs
pub struct CloseNonceBitmap<'info> {
    #[account(
        mut,
        seeds = [NONCE_BITMAP_SEED, eth_address.as_ref(), &word_pos.to_le_bytes()],
        bump = nonce_bitmap.bump,
        close = rent_payer,
    )]
    pub nonce_bitmap: Account<'info, NonceBitmap>,
    // ...
    pub rent_payer: UncheckedAccount<'info>,
}
```

Remediation

Remove `close_nonce_bitmap` and `CloseNonceBitmap` so finalized nonce state cannot be deleted. If rent reclamation is required, persist a non-closable finalization PDA keyed by `(eth_address, word_pos)` and require `use_unordered_nonce` and `invalidate_unordered_nonces` to reject finalized words before bitmap initialization.